Informe Técnico: Integración y Despliegue Continuo CI/CD (Punto 8)

Proyecto: SmartGymBro (GymBro AI)
Responsable DevOps: Aldo Dante Antivilo
Herramienta: GitHub Actions (Cloud Runner)
Archivo: .github/workflows/ci-cd.yml

1. Justificación y Objetivos de CI/CD

El **Punto 8** exige automatizar la construcción, verificación y publicación del software mediante un pipeline de CI/CD, garantizando que ninguna modificación defectuosa o vulnerable llegue a producción sin superar validaciones automáticas. Se seleccionó **GitHub Actions** por su integración nativa con el repositorio y su soporte para runners aislados en contenedores.

2. Disparadores del Pipeline (Triggers)

El workflow reacciona automáticamente ante los siguientes eventos de Git:

- **Push:** En ramas `develop`, `main` y ramas de características (`feature/**`).
- **Pull Requests:** En cualquier solicitud de integración hacia `develop` o `main`.
- **Release Tags:** Al etiquetar una versión de producción (`v*.*.*`).

3. Fases del Pipeline de Automatización

Fase	Comandos Ejecutados	Propósito y Criterio de Aprobación
1. Linters y Formato	`npm run lint`	Analiza el código estáticamente para detectar malas prácticas y patrones no permitidos.
2. Verificación de Tipos	`npm run typecheck`	Compila TypeScript en modo verificación estricta para asegurar consistencia de tipos en Angular y Backend.
3. Pruebas Unitarias	`npm test`	Ejecuta suites de prueba sobre la lógica de cálculo de 1RM, validación de descansos y bienvenida.
4. Compilación de Artefactos	`npm run build`	Compila los módulos de producción para confirmar que no existan errores de empaquetado.
5. Construcción Docker	`docker build` (Frontend y Backend)	Construye las imágenes oficiales verificando que los Dockerfiles compilen de forma hermética.

4. Política de Seguridad y Gestión de Secretos

> 🔒 **PROTECCIÓN DE SECRETOS EN EL RUNNER CLOUD:** Siguiendo las exigencias de ciberseguridad del TP, el pipeline **no contiene credenciales ni tokens en texto plano**. Se configuraron **GitHub Secrets** para inyectar variables sensibles durante el build sin exponerlas en el código fuente. Además, el pipeline tiene permisos restringidos (`permissions: contents: read`) aplicando el principio de menor privilegio.

5. Procedimiento Documentado de Rollback (Reversión)

La consigna exige definir un mecanismo de recuperación ante fallos en producción:

1. **Detección de fallo:** Si una versión desplegada genera alertas críticas en Prometheus o excepciones en la API.
2. **Rollback a nivel de Git:** Se revierte el commit culpable mediante `git revert <commit_hash>` en la rama `develop` o `main`.
3. **Rollback a nivel de Contenedores Docker:** Si la nueva imagen falla, se retrotrae al tag previo de la imagen mediante Docker Compose:

   ```
   docker compose stop backend frontend
   docker compose up -d --no-build backend frontend
   ```

4. **Rollback de Base de Datos:** Si el fallo involucró una migración corrupta, se utiliza el dump automático generado en `postgres/backups/` restaurándolo con `pg_restore` en el contenedor primario.

6. Evidencias de Ejecución Exitosa

> ✔️ **PIPELINE VALIDADO:** El pipeline ha sido verificado en GitHub Actions. Los jobs de *Lint*, *Typecheck*, *Unit Tests* y *Docker Builds* finalizan con estado verde (**Success**), impidiendo el merge de Pull Requests con errores.